

1. Logika cyfrowa

| Układ              | Formuła / sygnatura                                                 | Opis                                                                                                           |
|--------------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Sumatory           |                                                                     |                                                                                                                |
| Półsumator         | $s = x \oplus y$<br>$c = x \& y$                                    | Dodaje dwa bity. <b>s</b> to suma, <b>c</b> to bit przeniesienia (carry).                                      |
| Pełny sumator (FA) | $s = x \oplus y \oplus ci$<br>$co = x \& y \vee ci \& (x \oplus y)$ | Jak półsumator, ale przyjmuje też przeniesienie wejściowe <b>ci</b> .                                          |
| Sumator n-bitowy   | $n \times FA$                                                       | Kaskada (ripple-carry): wyjście $C_i$ trafia na wejście kolejnego FA. Ostatnie <b>co</b> to Carry Out całości. |

|                     |                         |                                                                                                                     |
|---------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------|
| Układy kombinacyjne |                         |                                                                                                                     |
| Dekoder             | $n \rightarrow 2^n$     | Wejście koduje liczbę $k$ . Na wyjściu zapalony jest <b>tylko</b> bit nr $k$ .                                      |
| Multiplekser        | $2^n + n \rightarrow 1$ | $2^n$ bitów danych + $n$ bitów sterujących $S$ . Na wyjście przechodzi $S$ -ty bit danych.                          |
| ALU                 | $A, B, f \rightarrow C$ | Dekoder na bitach $f$ wybiera operację (np. $A + B$ , $A B$ , $A \& B$ ); multipleksowanie wyników bramkami AND/OR. |

|                             |                                  |                                                                                                                                                    |
|-----------------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Układy sekwencyjne (pamięć) |                                  |                                                                                                                                                    |
| Przerzutnik S-R             | <b>S</b> - set, <b>R</b> - reset | Przechowuje 1 bit ( $Q$ ). <b>S=1</b> ustawia $Q = 1$ , <b>R=1</b> zeruje, <b>S=R=0</b> trzyma stan. Zbudowany z dwóch sprzężonych <b>NOR</b> -ów. |
| Sterowany poziomem          | aktywny gdy <b>CLK</b> = 1       | Wejścia <b>S</b> / <b>R</b> bramkowane AND-em z zegarem. Stan zmienia się tylko przy <b>CLK=1</b> .                                                |
| Sterowany zboczem           | zbocze 1 → 0                     | Dwa przerzutniki poziomowe w kaskadzie (master-slave), drugi z zanegowanym zegarem. Stan przepisuje się w <b>momencie</b> zmiany zegara.           |

2. Bity, bajty i typy danych

| Typ    | char | short | int | long | float | double | void* |
|--------|------|-------|-----|------|-------|--------|-------|
| x86-64 | 1    | 2     | 4   | 8    | 4     | 8      | 8     |

3. Kodowanie liczb i konwersja

$$B2U(X) = \sum_{i=0}^{w-1} x_i 2^i \quad B2T(X) = -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$$
$$T2U(x) = x < 0 ? x + 2^w : x \quad U2T(u) = u > TMax ? u - 2^w : u$$

**Skróty:** **B** = bity, **U** = unsigned, **T** = ze znakiem (kod U2). Stąd **B2U** = bity → unsigned, podobnie **U2T**, **UMax**, **TMax**, **UAdd**, **TAdd**.

| Stała       | Bity          | Wartość                          |
|-------------|---------------|----------------------------------|
| <b>UMax</b> | <b>11...1</b> | $2^w - 1$                        |
| <b>TMax</b> | <b>01...1</b> | $2^{w-1} - 1$ ( <b>INT_MAX</b> ) |
| <b>TMin</b> | <b>10...0</b> | $-2^{w-1}$ ( <b>INT_MIN</b> )    |
| <b>-1</b>   | <b>11...1</b> | te same bity co UMax             |

**Promocja w C:** **unsigned** i **int** w jednym wyrażeniu/porównaniu ( $< > ==$ ) → **int** promowany do **unsigned** (bity bez zmian,  $-1 \rightarrow UMax$ ).

4. Rozszerzanie, obcinanie i przesunięcia

|                                       |                                               |
|---------------------------------------|-----------------------------------------------|
| <b>Rozszerz. 0</b> ( <b>zext</b> )    | <b>unsigned</b> : dopisuje 0 z góry           |
| <b>Rozszerz. znak</b> ( <b>sext</b> ) | <b>signed</b> : powiela bit znaku (MSB)       |
| <b>x &lt;&lt; k</b>                   | $x \cdot 2^k$ (sgn./uns.)                     |
| <b>u &gt;&gt; k</b>                   | $\lfloor u/2^k \rfloor$ — logiczne (zera)     |
| <b>x &gt;&gt; k</b>                   | arytmetyczne (kopia MSB), zaokr. do $-\infty$ |
| $x/2^k$ do 0                          | <b>(x + (1&lt;&lt;k)-1) &gt;&gt; k</b>        |
| Strength red.: <b>x*24</b>            | → <b>(x&lt;&lt;5)-(x&lt;&lt;3)</b> .          |

5. Operacje, overflow i endianness

|                                        |                                                    |
|----------------------------------------|----------------------------------------------------|
| <b>UAdd</b>                            | $(u + v) \bmod 2^w$ ; carry ignorowane             |
| <b>TAdd</b>                            | bitowo = UAdd; różne znaki nie przepełniają        |
| <b>Nadmiar (+)</b>                     | $u, v > 0$ , a wynik $< 0$ (wynik $-2^w$ )         |
| <b>Nadmiar (-)</b>                     | $u, v < 0$ , a wynik $\geq 0$ (wynik $+2^w$ )      |
| <b>Negacja U2</b>                      | <b>-x = ~x+1</b> ; <b>-TMin=TMin</b> , <b>-0=0</b> |
| <b>Wykr. nadmiaru</b> ( <b>s=x+y</b> ) | <b>((s^x)&amp;(s^y))&gt;&gt;(w-1)</b>              |
| <b>Maska / abs</b>                     | <b>m=x&gt;&gt;31; abs=(x^m)-m</b>                  |

**Pamięć:** adres wskazuje najmłodszy bajt słowa. Napis = **char[]** ASCII + **0x00** (niezależny od endianness).

| Schemat              | Najniższy adres | <b>0x0123</b>    |
|----------------------|-----------------|------------------|
| <b>Big Endian</b>    | MSB             | <b>[01][123]</b> |
| <b>Little Endian</b> | LSB             | <b>[23][01]</b>  |

**Uwaga o sufiksach:** Instrukcje przyjmują sufiks określający rozmiar danych: **b** (8-bit), **w** (16-bit), **l** (32-bit), **q** (64-bit).  
Np. **movl** kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).